

LLM 모델선정, 지식베이스및RAG 구축

- RAG의 개념과 필요성
- LLM 모델 선정 기준과 절차
- 임베딩 모델 선정
- 지식베이스 데이터 수집과 정제
- 문서 청킹과 메타데이터 설계
- 벡터 DB와 유사도 검색
- 검색 품질 향상 (하이브리드 검색, 리랭킹)
- 프롬프트 구성과 환각 방지

1. RAG의 개념

RAG 개념: LLM이 답을 만들기 전에 학습 데이터 밖의 신뢰할 만한 지식베이스를 참조하게 해서 출력을 개선하는 방식, 비용 면에서도 효율적인 접근

2. LLM 모델 선정 기준과 절차

기준: 기능, 속도, 비용, effort 설정 (한 모델 안에서 지능과 지연 및 비용을 조절)

- **사용 사례 및 업무 적합성:** 번역, 요약, 코드 생성, RAG(검색 증강 생성), 도구 호출(Tool Calling) 등 실제 수행해야 할 작업에 필요한 성능과 도구 연동 능력을 갖췄는지 확인
- **한국어 및 도메인 품질**: 한국어 이해 및 생성 능력, 그리고 기업/산업 전문 용어(금융, 의료, 법률 등)의 정확도를 평가
- **보안 및 데이터 주권**: 사내 데이터를 클라우드 API에 전송해도 되는지, 아니면 폐쇄망 및 온프레미스(로컬) 환경에서 독립적으로 구동해야 하는지 결정
- **비용 효율성**: API 토큰 비용 또는 로컬 구동을 위한 하드웨어(GPU/VRAM) 구축·운영 비용을 고려하며, 무조건 가장 크고 비싼 모델이 아닌 업무에 최적화된 체급을 선택

- **지연 시간(Latency) 및 응답 속도**: 실시간 응답이 필요한 서비스인지, 배치 처리가 가능한지 등 요구되는 속도 기준을 충족하는지 확인

LLM 모델 선정 절차

1. **용도 및 요구사항 정의**
   - 어떤 팀이 어떤 업무에 사용할지, 동시 사용자 수, 보안 등급, 요구 응답 속도를 정리합니다.
2. **후보 모델 필터링 및 기초 평가**
   - 상용 모델(OpenAI, Anthropic 등)과 오픈소스/로컬 모델(Llama, Qwen 등) 중 요구 조건에 맞는 후보군을 추립니다.
   - 공개된 벤치마크 점수와 리더보드를 참고하여 1차 후보군을 압축합니다.
3. **자체 평가 세트(Evaluation Dataset) 구성**
   - 일반적인 벤치마크 대신, **실제 기업/프로젝트의 업무 데이터와 테스트 문항**으로 구성된 자체 평가 세트를 만듭니다.
   - 합격선(기준 점수)을 설정합니다.
4. **실증 테스트 및 비교**
   - 후보 모델들에 동일한 테스트용 시험지(평가셋)를 돌려보고, 품질, 토큰 소모량, 속도, 비용을 비교·분석합니다.
5. **최종 모델 선정 및 배포·모니터링**
   - 테스트 결과를 바탕으로 최적의 모델을 확정하고, API 연동 또는 로컬 인프라(VRAM/양자화 설정 등)에 배포한 뒤 지속적으로 성능을 모니터링합니다.

3. 임베딩 모델 선정

임베딩 유형

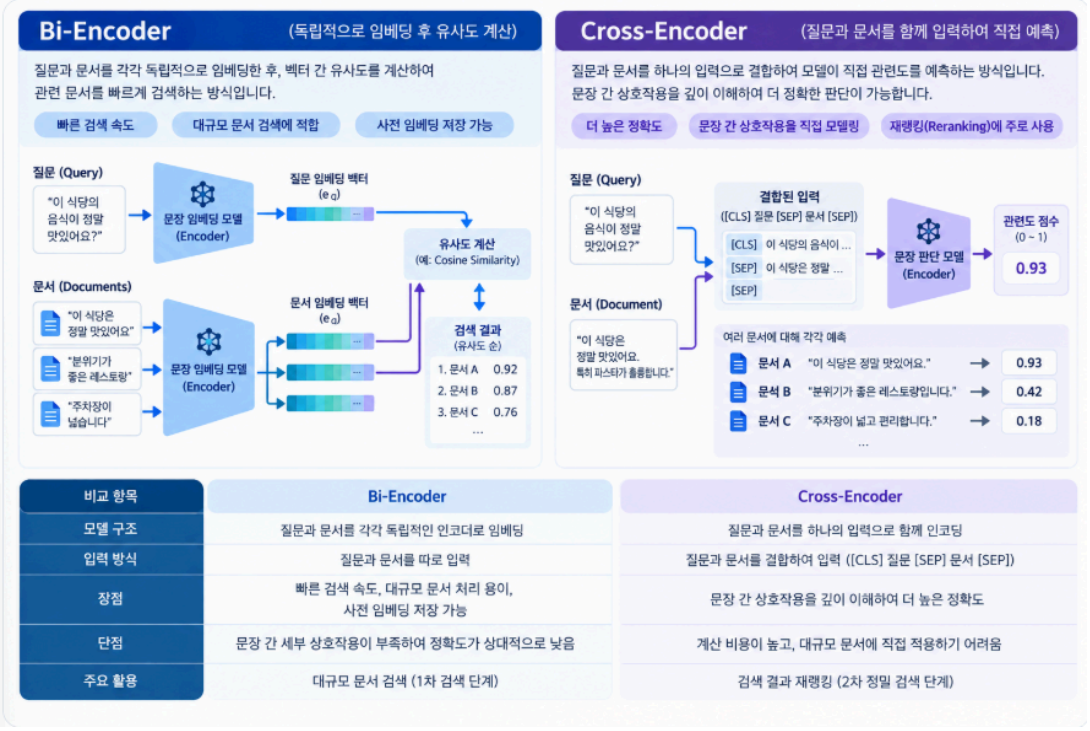
유형	단위	차원	사용 사례
단어 임베딩	단어	100~300	Word2Vec, GloVe (레거시)
문장 임베딩	문장/단락	384~4096	검색, 유사도 계산, RAG
문서 임베딩	전체 문서	768~4096	문서 분류, 클러스터링
멀티모달 임베딩	텍스트+이미지	512~1024	CLIP, 이미지 검색

임베딩 차원의 의미

차원	의미	장점	단점
384	경량	빠른 검색, 적은 스토리지	표현력 제한
768	표준	균형 잡힌 성능/비용	-
1024	고성능	높은 표현력	스토리지/비용 증가
1536	OpenAI 표준	매우 높은 표현력	높은 비용
3072	최대	최고 수준 정밀도	매우 높은 비용
4096	초대형	미세한 의미 차이 포착	검색 속도 저하

임베딩 모델 아키텍처

구분	Bi-Encoder	Cross-Encoder
구조	텍스트 A, B를 각각 독립 인코딩	텍스트 A+B를 함께 인코딩
출력	각각의 벡터 → 유사도 계산	직접 유사도 점수 출력
속도	빠름 (사전 인코딩 가능)	느림 (매번 쌍으로 계산)
정확도	중간~높음	매우 높음
용도	검색 (1차 검색)	리랭킹 (2차 정밀 평가)
스케일	수백만 문서 검색 가능	수십~수백 문서 비교



모델별 특징 상세

모델	장점	단점	적합한 경우
OpenAI text-embedding-3 시리즈	높은 범용 성능, 간편한 API, Matryoshka 임베딩 지원(차원 축소 가능)	API 비용 발생, 데이터 외부 전송, 오프라인 사용 불가	빠른 구현이 필요하고 API 비용을 감당할 수 있는 프로젝트
gemini-embedding-2 (Google)	텍스트·이미지·오디오·비디오·PDF를 단일 벡터 공간에 매핑하는 네이티브 멀티모달 지원, 100개 이상 언어, 128~3072차원 MRL	상용 API 중 높은 텍스트 단가, 최대 8192토큰	이미지·차트·스캔본이 포함된 문서의 멀티모달 RAG
Cohere embed-v4.0	128K 토큰 컨텍스트, 텍스트+이미지 동시 처리, int8/binary 양자화 내장, input_type으로 쿼리/문서 구분	상용 API, 오프라인 사용 불가	청킹 없이 긴 문서를 통째로 임베딩해야 하는 경우
Qwen3-Embedding 시리즈 (Alibaba)	0.6B/4B/8B 크기 선택, 32K 컨텍스트, 32~4096차원 MRL, 태스크별 instruction 지원, Apache 2.0	8B는 GPU 필수, instruction 프롬프트 설계에 따라 성능 편차	최고 수준의 다국어 오픈소스 성능이 필요할 때(0.6B는 경량 대안)
BGE-M3 (BAAI)	Dense + Sparse + ColBERT 멀티 검색, 다국어 강점, 한국어 파생 모델 생태계가 두터움	모델 크기 대비 추론 속도가 다소 느림	단일 모델로 하이브리드 검색을 구현하고 싶은 경우
KURE (고려대 NLP&AI Lab)	한국어 검색 특화, MTEB-ko-retrieval에서 BGE-M3 계열을 상회, MIT 라이선스	한국어 전용이라 다국어 서비스에 부적합; KURE-v2는 late-interaction 방식으로 벡터 DB 연동 방식이 다름	한국어 사내 문서 검색 등 언어가 한국어로 고정된 RAG
multilingual-e5-large	경량(560M), 다국어 균형 성능, CPU에서도 실행 가능	최대 토큰이 짧음(512)	리소스가 제한된 환경의 다국어 범용 검색

4. 지식베이스 데이터 수집과 정제

지식베이스 데이터 수집

수집 대상 데이터 유형

◦ **비정형 데이터:** PDF, Word(docx), Markdown, HTML, 사내 Wiki, 이메일, 상담 로그 등 (텍스트 중심)

◦ **정형/반정형 데이터:** 사내 DB(RDB, NoSQL), CSV, JSON, API 연동 데이터

핵심 고려사항

◦ **대표성(Representativeness):** 도메인의 지식을 충분히 포괄하고 있는가?

◦ **최신성 및 지속성:** 데이터가 주기적으로 업데이트되거나 실시간 동기화(파이프라인 구축)가 가능한가?

데이터 정제 및 전처리

텍스트 추출 및 파싱 (Parsing): PDF나 스캔 문서 내의 표(Table), 이미지, 레이아웃을 누락 없이 정확한 텍스트 문맥으로 변환합니다.

노이즈 제거 (Noise Removal): 의미 없는 특수문자, HTML 태그, 깨진 인코딩 문자, 중복된 공백 및 줄바꿈을 제거합니다.

중복 제거 (Deduplication): 지식베이스 내 동일하거나 유사한 문서가 중복 저장되어 검색 효율이 떨어지는 것을 방지합니다.

텍스트 정규화 (Normalization): 띄어쓰기, 맞춤법 교정 및 약어/유의어 처리(동의어 사전 정의 등)를 통해 텍스트의 일관성을 확보합니다.

5. 문서 청킹과 메타데이터 설계

RAG 시스템에서 문서 청킹과 메타데이터 설계는 검색 정확도와 답변 품질을 좌우하는 핵심 요소

문서 청킹 : 청킹은 긴 문서를 AI 모델이 처리하기 좋고 의미가 손실되지 않는 적절한 크기로 쪼개는 과정

청킹 방식

- **고정 길이 분할 (Fixed-size Chunking):** 지정된 글자수나 토큰 수로 자르는 방식입니다. 구현이 쉽지만 문장이나 의미가 중간에 끊길 수 있습니다.

- **구조 기반 / 의미 단위 분할 (Structure-aware / Semantic Chunking):** 문서의 목차(H1, H2, H3), 섹션, 표, 코드 블록, 문단 경계를 기준으로 나누는 방식입니다. 맥락적 무결성이 높습니다.
- **재귀적 분할 (Recursive Chunking):** 큰 단위(예: 섹션)로 먼저 나누고, 크기가 초과하면 더 작은 단위(문단, 문장)로 재귀적으로 쪼개는 방식입니다.

청크 오버랩

분할된 청크들이 앞뒤 내용과 단절되지 않도록 일정 부분(예: 10~20% 또는 50~100자)을 겹쳐서 저장합니다. 경계면의 맥락 유지를 돕습니다.

메타데이터 설계

청크만 단독으로 떨어져 있으면 이 내용이 어디서 온 것인지 맥락을 잃기 쉽습니다. 이를 보완하기 위해 각 청크에 구조화된 메타데이터를 함께 태깅해야 합니다.

필수 포함 메타데이터 항목:

- **출처 정보:** 문서 제목 (`title`), 원본 파일명, 페이지 번호, 섹션/목차 경로 (`hierarchy` 또는 `breadcrumb`).
- **시간 정보:** 작성일자, 수정일자, 버전 정보 (최신 근거 필터링용).
- **분류 정보:** 작성 부서, 도메인, 문서 유형(매뉴얼, 규정, FAQ, 코드 등).
- **요약/컨텍스트 주입 (Contextual Retrieval):** 문서 전체의 핵심 `description`이나 상위 요약문을 각 청크의 메타데이터나 텍스트 전면에 함께 결합해 주는 방식이 최근 검색 품질 향상에 매우 효과적입니다.

6. 벡터 DB와 유사도 검색

벡터 데이터베이스: 텍스트, 이미지, 오디오 등 비정형 데이터를 인공지능 모델을 통해 고차원의 숫자 배열인 벡터 임베딩(Vector Embedding)으로 변환하여 저장하고 관리하는 특수 목적의 데이터베이스

유사도 검색(Similarity Search): 단순히 정확한 단어가 일치하는 키워드 검색과 달리, 데이터가 가진 **의미론적(Semantic) 유사성**을 기반으로 쿼리 벡터와 가장 가까운 벡터들을 찾아내는 핵심 기술

벡터 DB의 작동 방식 (3단계)

데이터 임베딩 (Embedding)

◦ 텍스트나 이미지 같은 비정형 데이터를 AI 모델(BERT, CLIP 등)에 입력하여 의미를 담은 고차원 벡터로 변환합니다.

◦ 예: "고양이"와 "새끼 고양이"는 의미가 비슷하므로 다차원 공간에서 서로 가까운 좌표에 위치하게 됩니다. [1

저장 및 인덱싱 (Indexing)

◦ 수많은 벡터 데이터를 고속으로 탐색하기 위해 특수 인덱스 알고리즘을 사용합니다.

◦ 대표적인 인덱싱 기법:

▪ **HNSW (Hierarchical Navigable Small World)**: 그래프 기반의 근사 최인접 이웃(ANN) 검색 알고리즘

▪ **IVF (Inverted File Index)**: 공간을 여러 구역으로 나누어 클러스터링하는 방식

▪ **PQ (Product Quantization)**: 벡터를 압축하여 메모리 사용량을 줄이는 방식

유사도 계산 (Distance / Similarity Metrics)

• 쿼리 벡터와 DB에 저장된 벡터 간의 거리를 계산해 가장 유사한 순서대로 정렬합니다.

7. 검색 품질 향상

벡터 DB 기반의 유사도 검색에서 **검색 품질을 향상(Retrieval Quality Optimization)**시키기 위해서는 단순히 데이터의 의미적 유사성만 비교하는 것을 넘어, 데이터의 가공 단계부터 검색 후 순위 재조정까지 전 과정의 파이프라인을 고도화해야 합니다.

---

1. 청킹(Chunking) 전략 최적화

텍스트 데이터를 벡터로 변환하기 전, 적절한 크기로 자르는(Chunking) 단계가 매우 중요합니다. 너무 작으면 문맥이 잘리고, 너무 크면 노이즈가 섞여 의미가 희석됩니다.

- **재귀적 청킹 (Recursive Chunking):** 문단, 문장, 단어 단위로 텍스트 구조를 계층적으로 인식하여 의미적 단절 없이 자연스럽게 자릅니다.
- **청크 중첩 (Overlap):** 청크와 청크 사이에 일정 부분(예: 10~20%) 텍스트를 겹치게 하여 문맥이 잘리는 현상을 방지합니다.
- **상위-하위 청킹 (Parent-Child Chunking):** 실제 검색은 작은 단위(Child)의 의미로 수행하고, LLM에 컨텍스트를 전달할 때는 해당 내용을 포함하는 더 큰 단위(Parent)의 문맥을 복원하여 제공합니다.

2. 하이브리드 검색 (Hybrid Search)

벡터 검색(밀집 검색, Dense Retrieval)은 문맥적 의미를 잘 잡지만, 품번, 고유 명사, 날짜 등 **정확한 키워드 매칭**에는 약합니다. 이를 보완하기 위해 전통적인 키워드 검색과 결합합니다.

- **비율 기반 결합 (RRF - Reciprocal Rank Fusion):** 전통적인 키워드 검색 알고리즘(**BM25**)의 순위 결과와 벡터 검색 알고리즘의 순위 결과를 수학적으로 조합하여 가장 균형 잡힌 결과를 도출합니다.

3. 리랭킹 (Reranking) 후처리

벡터 검색은 대규모 데이터에서 '빠르게 후보군(Top-K)'을 추려내는 데 특화되어 있지만, 미세한 순위 정확도는 떨어질 수 있습니다.

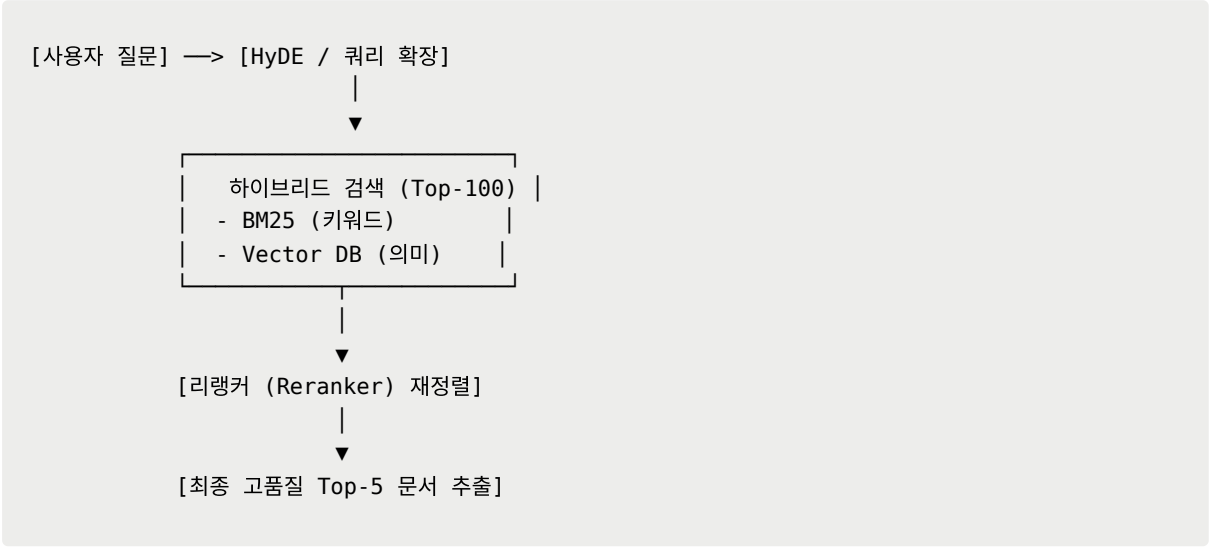
- **교차 인코더(Cross-Encoder) 활용:** 1차 벡터 검색으로 상위 50~100개의 결과를 뽑아낸 뒤, 훨씬 정교하고 무거운 AI 모델인 **리랭커(Reranker, 예: BGE-Reranker, Cohere Reranker 등)**를 통해 사용자의 질문과 문서 간의 연관성을 다시 정밀 검사하여 최종 순위를 재정렬합니다. 이 단계를 거치면 검색 정확도가 대폭 상승합니다.

4. 쿼리 변형 및 확장 (Query Transformation)

사용자가 입력한 질문이 모호하거나 너무 짧으면 검색 품질이 떨어집니다. 사용자의 질문을 검색하기 좋은 형태로 다듬는 기법입니다.

- **가상 문서 임베딩 (HyDE - Hypothetical Document Embeddings):** 사용자의 질문을 바탕으로 LLM이 '가상의 답변 문서'를 먼저 작성하게 한 뒤, 그 가상 답변을 벡터로 변환하여 DB를 검색합니다. 질문과 문서를 비교하는 것보다 **답변과 문서를 비교할 때 유사도가 더 정확하게 측정**되는 원리를 이용합니다.
- **하위 질문 생성 (Query Decomposition):** 복잡한 질문을 여러 개의 단순한 질문으로 쪼개어 각각 검색한 뒤 결과를 종합합니다.

예시



8. 프롬프트 구성과 환각 방지

1. 환각 방지를 위한 프롬프트 구조화 (Prompt Engineering)

LLM에게 자유도를 주지 않고, **"제공된 문서 안에서만 답변하라"**는 명확한 제약 조건을 프롬프트에 부여해야 합니다. 이를 위해 **역할 정의, 컨텍스트 제공, 제약 조건, 출력 형식**을 명확히 구분하는 구조적 프롬프트가 필요합니다.

2. LLM 에서의 환각 방지 기술적 설정

프롬프트 엔지니어링 외에도 LLM의 API 설정과 후처리를 통해 환각을 물리적으로 차단할 수 있습니다.

- **온도(Temperature) 및 Top-P 최적화:**
  - LLM의 `temperature` 값을 **0에 가깝게(예: 0.0 ~ 0.2)** 설정합니다. 온도가 낮을수록 모델은 창의성을 버리고 가장 확률이 높고 확실한 단어만 선택하므로 환각이 급격히 줄어듭니다.
- **인용(Citation) 및 출처 표기 강제:**
  - 모델에게 답변의 근거가 된 문서 번호나 문장을 함께 출력하도록 강제합니다. 사용자가 직접 팩트 체크를 할 수 있게 만들 뿐만 아니라, LLM 스스로도 출처를 매핑하는 과정에서 이상한 답변을 할 확률이 낮아집니다.
- **가드레일(Guardrails) 도입:**
  - **NeMo Guardrails**나 **Llama Guard** 같은 도구를 파이프라인 마지막 단계에 배치합니다. LLM이 내놓은 최종 답변과 벡터 DB에서 뽑아온 원본 문서를 비교하여, 원본에 없는 단어나 내용이 포함되어 있다면 답변을 차단하거나 수정하는 필터링 역할을 합니다.

3. 데이터 및 검색 단계에서의 연계 조치

아무리 프롬프트를 잘 짜도 벡터 DB가 잘못된 문서나 오염된 데이터를 주면 LLM은 환각을 일으킵니다.

- **컨텍스트 순서 배치 (Lost in the Middle 방지):**
  - LLM은 컨텍스트의 맨 앞과 맨 뒤에 있는 정보를 가장 잘 기억하고, 중간에 있는 정보는 놓치는 경향이 있습니다. 리랭킹(Reranker)을 통해 **가장 연관성이 높은 핵심 문서를 [컨텍스트]의 맨 앞과 맨 뒤에 배치**해야 환각이 줄어듭니다.
- **정확한 메타데이터 필터링:**
  - 구버전 문서와 신버전 문서가 섞여 있으면 모델이 엉뚱한 과거 정보를 정답으로 인지합니다. 날짜, 카테고리 등의 메타데이터 필터링을 벡터 검색 시 필수로 적용해야 합니다.